

想定Q&A — WireGuard for Apache NuttX (Community Over Code, Glasgow)

[coc-glasgow-slides.html](#)(カーネル移植後、26枚)用の Q&A。 [presentation-script.md](#) の想定Q&Aは古い [slides.html](#) (apps版のみ、FLATビルド前提)向けで、この版には合わない — 特に「PROTECTED/KERNELは 今後の課題」という回答はもう事実と異なる(検証済み)。このデッキで話す/ 質問を受けるときはこちらを使うこと。

各項目は「JAで理解する内容」+「実際に英語で言う一言」。根拠は [verification-matrix.md](#)、[handoff.md](#)、[tai64n-decision.md](#)、[open-questions.md](#)。「全部検証済み」「マージ可能」とは絶対に言わない — 検証マトリクスの書き方に合わせ、できたことと同じ重さでできていないことも言う。

設計・移植方針

Q. なぜ lwIP をそのまま NuttX に載せなかったのか? NuttX は独自のネットワークスタックを持っていて、そちらが標準です。lwIP を 別途載せると二重にスタックを持つことになり、メモリのにも保守的にも不利です。NuttX ネイティブの netdev として実装すれば `ifconfig` や既存のアプリが そのまま使えます。Say: "NuttX has its own network stack; adding lwIP alongside it means two stacks in memory and in maintenance. A native netdev gets you `ifconfig` and every existing app for free."

Q. apps/ 版とカーネル版、結局どちらが「本物」なのか? 両方とも本物で、目的が違います。apps版(v0.1.1、FLATビルド専用)は `psock_*`() という内部APIに依存していて、これは同一アドレス空間の FLATビルドでしか使えません。カーネル版は `drivers/net/wireguard/` に 住む正式な in-tree ドライバで、PROTECTED/KERNELビルドでも動きます。upstream に出すのはカーネル版です。apps版は「動くと分かっていたものを 土台に、どこを直せばカーネルに収まるか」を洗い出すために先に作った、という位置づけです。Say: "The apps version proved the protocol port and found five bugs on a FLAT build. The kernel driver is the one going upstream — it works under every privilege model, not just FLAT."

Q. 内部 API (`psock_*`()) を使うのは問題にならないのか? apps版ではそれが制約でした(FLATビルド専用)。カーネル版はもうこれに 依存していません — `wg0` はカーネル内の lower-half netdev で、ソケットも 受信スレッドもドライバ自身が所有し、ユーザー空間とは固定長・ポインタなしの `ioctl` ABI でやり取りします。これが PROTECTED/KERNEL ビルドで動く理由です。Say: "That was true of the apps version. The kernel driver doesn't touch internal socket APIs at all — it owns the socket itself and talks to user space through a flat, pointer-free `ioctl` ABI, which is why it runs under PROTECTED and KERNEL builds."

検証の範囲(スライド18・19の深掘り)

Q. 「7つの場所で検証した」というが、実機で足りていないものは何か? ESP32-S3 での資源実測(スタック高水位など)と、`esp32s3-devkit:knsh` (実シリコン上のPROTECTEDビルド)。今回計測できたのは SPRESENSE (Cortex-M4F)側で、どちらもボードを接続すればすぐ回せる形にスクリプト化 してありますが、まだ回せていません。ESP32-S3 自体でのトンネル動作(ネイティブWi-Fi、4/4成功)は検証済みで、資源実測だけが残っています。Say: "ESP32-S3 resource measurement and PROTECTED on that board's real silicon — both are scripted and ready, neither has run yet because the

board wasn't attached this cycle. The tunnel itself on ESP32-S3 is verified; it's the stack/heap numbers on that specific chip that are still open."

Q. IOB(パケットバッファ)が枯渇したらどうなるのか? プールを12バッファまで絞って、トンネル越しのトラフィックと偽の ハンドシェイク開始のフラッドを同時にかけ、枯渇させることは確認しています — assertも落ちず、負荷が止まればプールも通信も元に戻ります。「ドライバが実際にIOBの確保待ちでブロックし得る経路があるか」もその後 [job-wait-audit.md](#) で確定させました: バッファ付きUDP送受信の経路はどちらも try-allocation (`throttled=false / timeout=0`)なので、`nwait` が動く「確保待ちで タスクが眠る」経路はこのドライバには**存在しません** — 「未検証」ではなく「到達不能」です。usrsockのソケット自体がブロックする経路や AP喪失はこれとは別物で、混同しないようにしています。Say: "Shrinking the pool and flooding it survives — no assert, full recovery. Whether the driver can ever block waiting for a buffer is settled, not open: the direct TX/RX paths are try-allocation only, so there's no wait branch to exercise. That's 'not reachable,' which is a better answer than 'untested.'"

Q. #14(タイムスタンプ/リブレイ)はなぜまだ open なのか、証拠が 足りないのでは? 証拠が足りないからではなく、判断として open にしています。永続化した 範囲予約という「本来の」解決策は設計済みですが、実装していません — NuttXには「この書き込みは電源断を越えて残る」と約束するストレージの 契約がなく(`hostfs_sync()` はvoidの `host_sync()` を呼んでOKを返すだけ)、その上に作ると「解決したように見えて解決していない」ものになります。実機で対照実験もしています: ボードには何も送らず自分自身のセッション だけを見ると、時計が未設定なら75秒経ってもハンドシェイクが成立せず、時計を設定すると4.1秒で成立する — 変数は時計だけなので、これが原因だと 言い切れます。Say: "It's open by decision, not by lack of evidence. The durable answer needs a storage durability contract NuttX doesn't offer yet, so I won't ship something that only looks solved. I did measure the failure on real hardware with a controlled pair — clock unset, no handshake in 75 seconds; clock set, 4.1 seconds."

Q. その「TZ(鍵のゼロ化)」の検証で `next_keypair` やハンドシェイク状態が 非ゼロになる瞬間は見られたのか? その後見られるようになりました。UDPリレーで転送確認を意図的に止めて `next_keypair` (responderが次のセッションをインストールしてから現行に 昇格するまでの短い窓)を非ゼロのまま捕捉し、応答のない開始要求で ハンドシェイク状態(鍵導出の瞬間に消去される一時鍵・チェーン・ハッシュ)も非ゼロのまま捕捉しました。どちらも `wg down` 後はゼロです。タイマー 駆動の自然な再鍵化(ピア側のリセットなし)もこの間に確認しています。Say: "It's closed now — a UDP relay that withholds the transport confirmation catches `next_keypair` non-zero, and an unanswered initiation catches the handshake state non-zero. Both go to zero after `down`, on sim, under one controlled delivery schedule."

upstream・提出状況

Q. もうPRは出したのか? マージされているのか? まだupstream PRは出していません。リベースと再検証を済ませ、`crypto:` → ドライバ → apps の3本の署名済み候補branchを**自分のforkへpush済み**です。PRの作成と

dev@nuttx.apache.org への投稿はまだです。これは意図的な順序で、upstreamへの実際の投稿は自分の責任で行い、この場では「こう作った、こう検証した」という提出候補を示しています。Say: "Not upstream yet. The rebased and re-verified series is published as three signed candidate branches on my forks. The dev-list discussion and the actual upstream PRs are the next step."

Q. `checkpatch` や `nxstyle` は通っているのか? はい。ファイル単位の `nxstyle -f` に加え、CIがPRに対して行うパッチ形式の `checkpatch.sh -g <range>` も**3系列で実施済み**です。候補branchはその状態で 自分のforkへpushしています。Say: "Yes. Both per-file `nxstyle` and the patch-form `checkpatch.sh -g` checks pass for all three published candidate branches."

Q. これはGSoCプロジェクトだったのか? 出発点にはGSoC提案としての検討もありましたが、この発表で示すコードと検証は、採否とは切り離して継続したコミュニティ向けの開発成果です。だから発表の軸も 制度ではなく、実装、失敗、検証、upstream reviewに置いています。Say: "It began with work around a GSoC proposal, but the engineering continued independently. This talk is about the implementation, the evidence, and the upstream contribution, not the program outcome."

Q. NuttX自身のバグ(ChaCha20-Poly1305のnonce)は誰が直すのか? このプロジェクトの成果に含まれるのか? このプロジェクトの中で見つけたバグですが、直し方は独立したものです。カウンタ0では新旧のnonceレイアウトが一致するため、これまで誰も (in-treeの呼び出し元もなく)踏んだことがありませんでした。修正は 2行で、ドライバのPRとは別に、単独の crypto: PR として先に出す 予定です — それ単体で成り立ち、誰の役にも立つからです。Say: "Found while porting, but it's a genuine pre-existing NuttX bug, not something the driver introduced. Two-line fix, shipping as its own crypto: PR ahead of the driver PR."

性能・運用

Q. 性能は? ESP32-S3 実機・実 Wi-Fi 経由で、トンネル越しの TCP スループットが約 260 KiB/s です。7 MB の連続転送を問題なく通せています。ただしこれは「明らかなボトルネックを1つ潰した」段階の数字で、体系的なチューニングはしていません。暗号処理がソフトウェア実装のままなので、そこがおそらく 次のボトルネックです。Say: "About 260 KiB/s TCP through the tunnel on ESP32-S3 over real Wi-Fi, 7 MB transfers clean. That's after fixing one obvious bottleneck, not a tuned number — software crypto is probably next."

Q. 暗号処理はソフトウェア? ハードウェアアクセラレータは? 現状はNuttX自身のソフトウェア実装(BLAKE2s, ChaCha20-Poly1305, Curve25519)をそのまま使っています。cryptodev経由でCurve25519に 到達すること自体は技術的に可能です(CRK_DH_COMPUTE_KEY → swcr_dh_make_common)が、そのハンドラは CRYPTO_CRYPTODEV_SOFTWARE_CRYPTO 経由でしか有効にならず、それは CRYPTO_SW_AES に依存しています。つまり1回のスカラー倍算のために、ソフトウェア暗号スイート一式を イメージに引き込むことになり、割に合わないと判断して見送りました。Say: "Software throughout. Reaching curve25519 via cryptodev is technically possible but drags in the whole software cipher suite as a dependency price — not worth it for one scalar multiplication."

Q. 長時間の安定性は? apps版(ESP32-S3)では最長4時間28分の連続稼働を確認していて、止まった 原因はESP32-S3のWi-Fiドライバ側の問題でWireGuardとは無関係でした。カーネル版については、sim上で151回の強制再鍵化・10回のエンドポイント 移動・100回のdown/upを620秒で回し、リソースが起動時と完全に一致する ことを確認していますが、これはシミュレータです。実機側は SPRESENSEで120秒の負荷と40回のdown/upサイクルを確認済みで、こちらも 資源は前後で完全一致します。ただし「日単位で動く」ことはまだ実機の カーネル版では言えません。Say: "The apps version ran 4h28m on hardware before an unrelated Wi-Fi driver bug stopped it. The kernel driver's soak is 620 seconds of forced churn in the simulator, plus 40 down/up cycles measured on real SPRESENSE hardware — both come back to identical resource numbers. Day-scale uptime on the kernel driver, on hardware, isn't something I can claim yet."

Q. 複数ピアは? 対応済みです。CONFIG_NET_WIREGUARD_MAX_PEERS で1~16まで設定でき、シミュレータ上でLinuxカーネルのWireGuardを2本立てて、両方と同時に セッションを保持することを確認しています。ピアは静的に確保されるのでRAMと直接のトレードで、実測で1ピアあたり約904バイトです。実機での 複数ピア確認はまだです。Say: "Supported, 1 to 16 via Kconfig, verified with two simultaneous Linux peers in the simulator — about 904 bytes of RAM per peer, statically allocated. Not yet confirmed with multiple peers on hardware."

Q. 鍵の管理は? 実行時に設定できます。wg genkey で生成し、wg set private-key で 設定、wg saveconf で保存すると次回起動時に自動で読み込まれます。ビルド成果物に鍵を焼き込む必要はありません。設定ファイルの二重 書き込みで壊れるバグ(#17)を2件見つけて直しました — 同時に2つの 書き込みが起きると設定が消える経路があったのと、共有の一時ファイル名が 競合する経路です。SmartFS上での電源断耐性はまだ未検証です (切り替え可能な電源が必要)。Say: "Runtime genkey/set/saveconf, nothing baked into the build. Found and fixed two real bugs in the config-save path where concurrent writers could destroy the file. Power-cut durability on SmartFS specifically is still unverified — needs switchable power I didn't have this cycle."

Q. 設定ファイルの形式は独自? wg(8) と同じ INI 形式です([Interface] / [Peer]、PrivateKey / PublicKey / AllowedIPs / Endpoint / PersistentKeepalive)。デスクトップの .conf をそのまま持ち込めます。Say: "Same INI format as wg(8) — a desktop .conf drops in as-is."

ハードウェア選定

Q. なぜ ESP32-S3 と SPRESENSE を選んだのか? NuttXのWi-Fiサポートがあることが第一です。手元のESP32-WROOM-32は 書き込みに入れず(GPIO0のブートモード遷移まで切り分け済み)、SPRESENSEは オンボードWi-Fiがなく外付けのGS2200Mモジュール経由になるので、usrsock という別の経路(ボード側の socket() がWi-Fiモジュールに 渡る)も同時に検証できる利点がありました。Say: "NuttX Wi-Fi support was the filter. SPRESENSE's Wi-Fi is an off-chip GS2200M module through usrsock, which is a genuinely different code path from ESP32-S3's native Wi-Fi — so the two boards together cover two different integration styles, not just two chips."

Q. Raspberry Pi Pico 2 W は使えないのか? オンボードのWi-Fiチップ(CYW43439)のドライバが、NuttXではRP2040版にしか統合されていません。RP2350側にもPIO/GPIOの対応関数は揃っている ので移植自体は現実的ですが、別のドライバ開発タスクになるので今回は スコープ外としました。Say: "The onboard Wi-Fi chip's driver is only wired up for RP2040 in NuttX today, not RP2350 — porting it is plausible but a separate driver task, out of scope here."

バージョン依存

Q. NuttXのバージョンは何に依存しているのか? していません。12.7.0とmasterの両方でビルドが通ります (--build-arg NUTTX_REF=<ref>)。masterではCONFIG_NSH_LINELENが CONFIG_LINE_MAXに改名されており、古い名前を指定してもエラーにならないままwg set peerの行だけが切れる、という静かな壊れ方を 一度見つけて直しています。Say: "Builds clean on both 12.7.0 and master via a build arg. Found one silent breakage along the way — a renamed Kconfig symbol that truncated a config line without erroring — and fixed it."

台本と合わせて使う補足

- 質問が来る可能性が高いもの: upstream提出状況(fork push済み、PR/dev@は未実施)、ESP32-S3の資源実測待ち、#17のSmartFS電源断待ち。すべてこの資料で答えられるようにしてある。TZ・TIはどちらも もう閉じている(前者は測定、後者は「到達不能」の確認) — 古い 資料や記憶で「まだ狭い」と言わないこと。
- 逆にスライドに載っている数字を聞かれたら、スライド18・19・20を 直接指させばよい(URLハッシュ #18 などて即座に飛べる)。